

EE470 – Microcontrollers and Embedded Systems

Pre-Lab Report – Modified Experiment 3.5

Frequency Measurement using Interrupts Only (No Polling, No `_delay_ms()`)

Student Name: Ahmad Ali Masarrat

Student ID: 156672

Date: November 25th 2025

Objective

Modify Experiment 3.5 to measure the frequency of an unknown square wave applied to **T0 (PD4)** using **only interrupts**.

The measurement must be performed over an **exact 1-second gate time** and the result must be printed on the serial terminal every second.

The solution must correctly handle frequencies from **< 10 Hz** up to **> 300 kHz** without using any polling or delay functions.

Theory & Calculations ($F_{CPU} = 16 \text{ MHz}$)

Prescaler = 1024 $\rightarrow$ Tick period = $1024 / 16\,000\,000 = 64 \mu\text{s}$

1 second = $1 / 64 \mu\text{s} = 15\,625 \text{ ticks}$

Timer2 is 8-bit $\rightarrow$ maximum OCR2A = 255 $\rightarrow$ cannot produce 1 s directly.

so? used **CTC mode** with a software counter inside the ISR.

OCR2A = 124 $\rightarrow$ Period = $(124 + 1) \times 64 \mu\text{s} = 8 \text{ ms}$

Number of interrupts for 1 second: $15\,625 / 125 = 125 \text{ exactly}$ $\rightarrow$ Every 125th interrupt we capture the pulse count $\rightarrow$ **exact 1 second gate time**

Interrupt Only Code

C

/ exp3_5_interrupt_version.c*

EE470 - Experiment 3.5 modified with interrupts only

Frequency measurement using TC0 (external counter on T0/PD4)

1-second gate time generated by TC2 Compare Match A interrupt in CTC mode

**/*

```

#include "ee470avr.h"
#include <stdio.h>

volatile uint32_t total_pulses = 0;
volatile uint8_t update_freq = 0;

ISR(TIMER0_OVF_vect)
{
    total_pulses += 256;
}

ISR(TIMER2_COMPA_vect)
{
    static uint8_t ticks = 0;
    ticks++;

    // 125 interrupts × 125 ticks each = 15625 timer ticks = exactly 1 second
    if (ticks >= 125) {
        total_pulses += TCNT0;
        TCNT0 = 0;
        update_freq = 1;
        ticks = 0;
    }
}

int main(void)
{
    char line[80];

    bitclr(DDRD, PD4); // T0 input

    serial_init(103); // 9600 @ 16 MHz
    putstr("\r\nFrequency Meter (Interrupt-Driven)\r\n");

    TCCR0A = 0x00;
    TCCR0B = 0b111;
    TCNT0 = 0;
    bitset(TIMSK0, TOIE0);

```

```

TCCR2A = (1 << WGM21); // CTC
TCCR2B = 0b111;        // Prescaler 1024
OCR2A = 124;
bitset(TIMSK2, OCIE2A);

sei();

while(1)
{
    if(update_freq)
    {
        uint32_t pulses = total_pulses;
        total_pulses = 0;
        update_freq = 0;

        sprintf(line, "Frequency = %lu Hz\r\n", pulses);
        putstr(line);
    }
}
}

```

Register Summary Tables

Table 1 Timer/Counter0 (External Counter on T0/PD4)

Register	Bit(s)	Value	Purpose
DDRD	PD4	0	PD4 input
TCCR0A	–	0x00	Normal mode
TCCR0B	CS02:CS00	111	External clock on T0 rising edge
TIMSK0	TOIE0	1	Enable overflow interrupt
TCNT0	0–255	–	Current pulse count

Table 2 Timer/Counter2 (1-Second Gate Generator)

Register	Bit(s)	Value	Purpose
----------	--------	-------	---------

TCCR2A	WGM21	1	CTC mode – clear timer on compare
TCCR2B	CS22:CS20	111	Prescaler 1024
OCR2A	–	124	Compare value → 125 ticks = 8 ms
TIMSK2	OCIE2A	1	Enable Compare Match A interrupt

Table 3 Variables

Variable	Type	Scope	Purpose
total_pulses	volatile uint32_t	Global	Total pulses in current 1-second window
update_freq	volatile uint8_t	Global	Flag set every 1 second
ticks	static uint8_t	ISR	Counts 125 × 8 ms interrupts → exact 1 s

Flowchart

Start



Initialize UART, PD4=input, TC0 (external), TC2 (CTC 8ms), enable interrupts



sei() → global interrupts ON

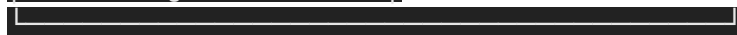


while(1)

if(update_freq) → Print frequency

Print frequency Reset flag & total_pulses

Clear flag & counter



TC0 OVF ISR TC2 COMPA ISR (every 8ms)

total_pulses += 256 ticks++

if ticks >= 125 → set flag, reset